

Docker

Instructor - <https://in.linkedin.com/in/suryakantsurve>

Course Outline

- What are Containers and Why Use Them?
- Differences Between Virtual Machines and Containers
- Introduction to Docker Architecture
- Docker CLI Basics: Images, Containers, Volumes, and Networks
- Creating and Managing Docker Containers
- Basic commands (docker run, ps, stop, rm)
- Pull and run Ubuntu/nginx containers
- Container lifecycle (create, start, stop, remove)
- Exec into a running container (docker exec)
- View logs (docker logs)
- Writing and Using Dockerfiles
- What is a Dockerfile?
- Key instructions: FROM, RUN, COPY, CMD, ENTRYPOINT, EXPOSE
- Building custom images (docker build)
- Building Custom Docker Images
- Docker Volumes and Persistent Data
- Docker Networking Essentials
- Docker Hub (push/pull images)
- Docker Compose: Multi-Container Applications

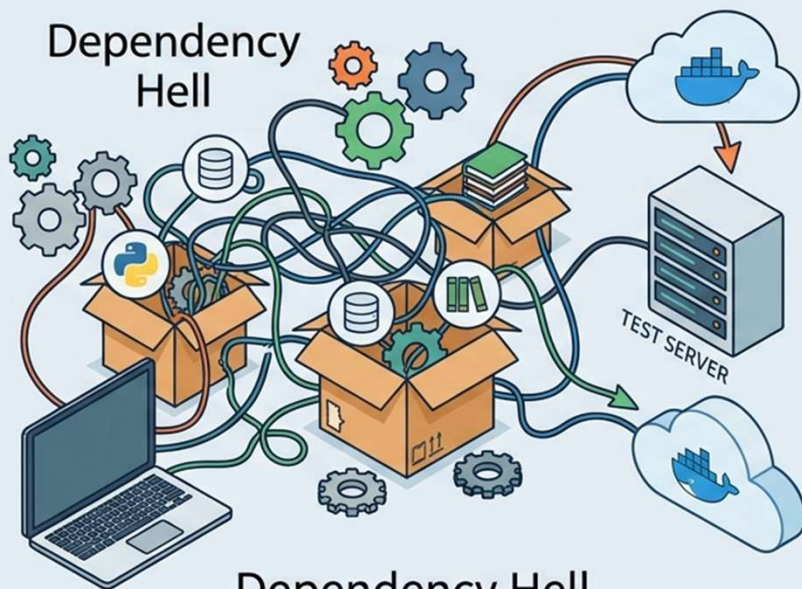
What is Docker?

- Docker is a platform used to Build , Package , Ship ,Run applications inside lightweight containers.
- Think of Docker as a **shipping container**.
- Just like shipping containers can transport goods anywhere, Docker containers transport applications anywhere.

The Docker Platform

- Docker provides the ability to package and run an application in a loosely isolated environment called a container.
- The isolation and security let you run many containers simultaneously on a given host.

BEFORE DOCKER



Dependency Hell
Inconsistent Environments
It works on my machine!

WITH DOCKER



Standardized
Packaging

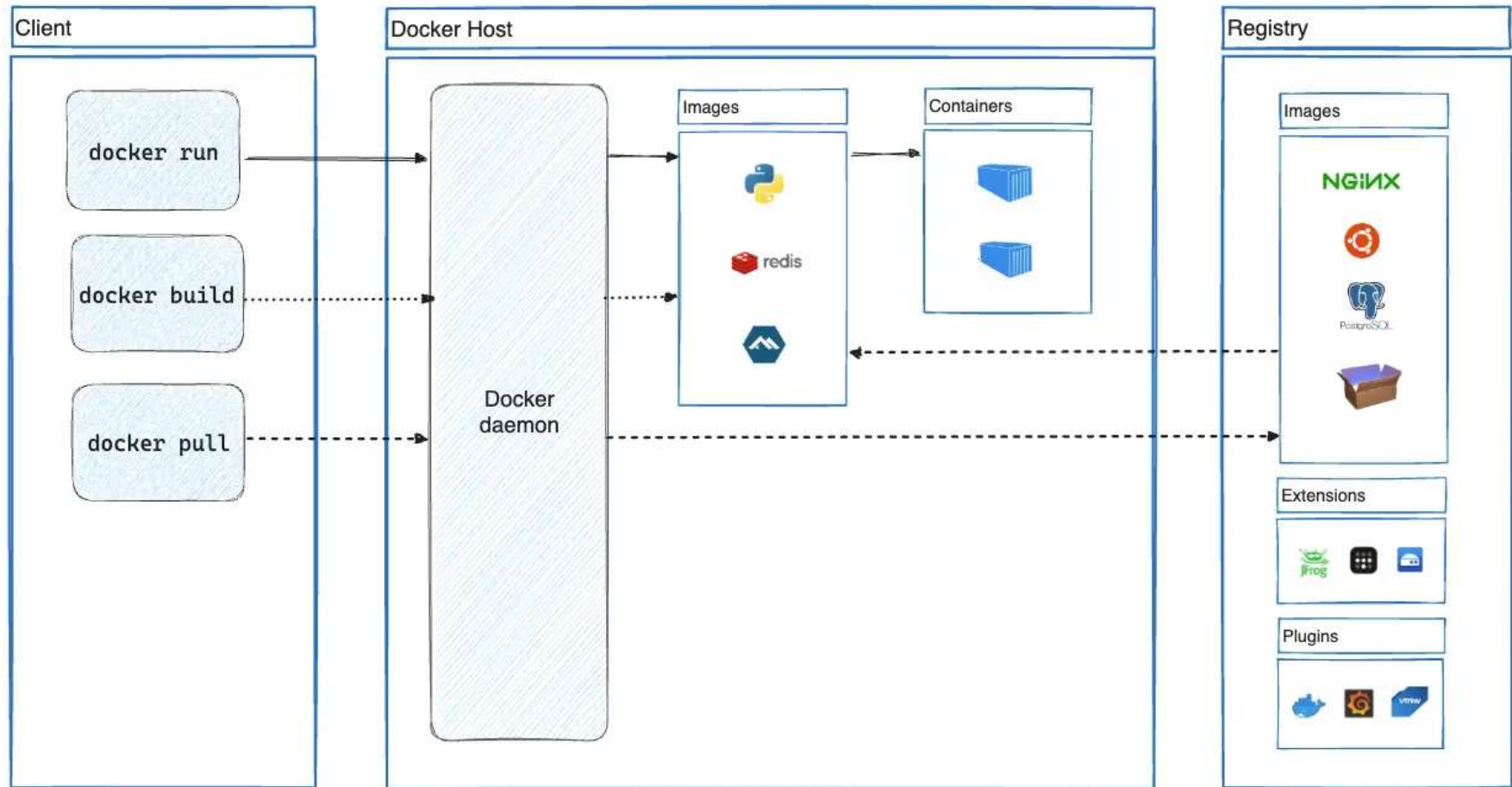


Standardized Packaging
Consistent Everywhere
Build Once, Run Anywhere

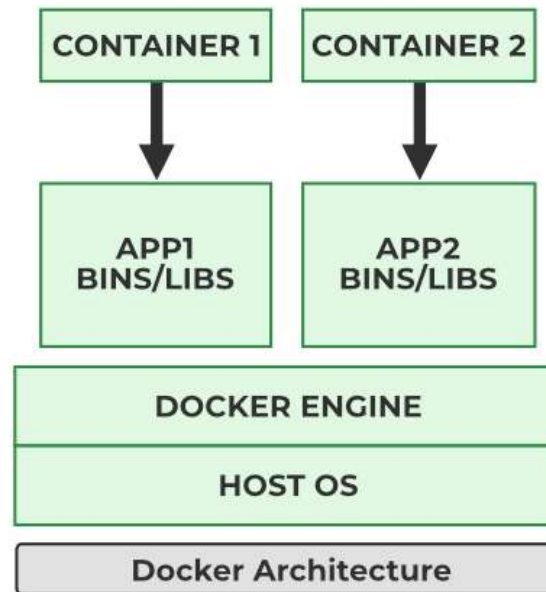
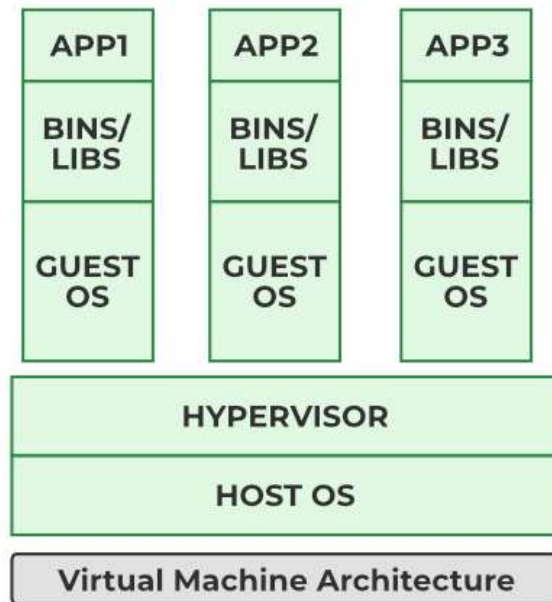
Docker Components

- Docker Image → Blueprint - ubuntu:22.04
- Docker Container → Running instance of an image
- Docker Hub → Online repository

Docker Architecture



Docker vs Virtual Machine



Docker vs Virtual Machine

Virtual Machine

Heavy

Boots in minutes

Own Operating System

More Memory

Large Storage

Docker

Lightweight

Starts in seconds

Shares Host OS

Less Memory

Small Images

First Docker Command

- `docker run hello-world`
- Docker automatically
 - Pulls image
 - Creates container
 - Runs container
 - Stops container

Docker Commands

- `docker pull ubuntu`
- `docker images`
- `docker run ubuntu`
- `docker run -it ubuntu bash`
- `docker run -d nginx`
- `docker ps`
- `docker ps -a`
- `docker stop container_id`
- `docker start container_id`
- `docker restart container_id`
- `docker rm container_id`
- `docker rmi image_name`
- `docker run --name mynginx nginx`
- `docker run -d -p 8080:80 nginx`
- `docker run -d -e MYSQL_ROOT_PASSWORD=admin mysql`
- `docker exec -it mynginx bash`
- `docker logs mynginx`

Docker Volumes

- Docker volumes are specialized directories managed directly by Docker that allow your containers to store and share data persistently.
- Because Docker containers are inherently ephemeral—meaning they erase all internal data when deleted—volumes serve as external "hard drives" that keep your data safe even if the container crashes or is completely removed.

Why You Need Volumes: The Problem

- Data Loss: If a database container is updated or deleted, all saved metrics or accounts disappear.
- Performance: Writing data directly into a container's default filesystem slows down efficiency due to storage driver translation overhead.
- Sharing: Multiple containers cannot easily access or edit the same files within another container's private directory.

How Docker Storage Types Compare

Storage Type 	Managed By	Where It Lives on Host	Best Used For
Named Volumes	Docker Engine	<code>/var/lib/docker/volumes/</code>	Databases and long-term production files.
Bind Mounts	You (Manually)	Any directory you choose	Live code-reloading during local development.
tmpfs Mounts	Host OS Memory	System RAM only	Sensitive tokens, secrets, or high-speed cache.

Essential Command Guide

- Create a volume: `docker volume create my_data`
- List active volumes: `docker volume ls`
- Inspect configurations: `docker volume inspect my_data`
- Delete a specific volume: `docker volume rm my_data`
- Purge all unattached volumes: `docker volume prune`

Container with Volume

Using the standard -v flag (VolumeName:ContainerPath)

```
docker run -d --name web_app -v my_data:/app/storage nginx
```

Using the more explicit --mount flag

```
docker run -d --name web_app --mount source=my_data,target=/app/storage nginx
```

MySQL Example

- **docker volume create mysql-data**
- **docker run -d --name mysql-db -e MYSQL_ROOT_PASSWORD=admin -v mysql-data:/var/lib/mysql mysql**

Where Are Volumes Stored?

`/var/lib/docker/volumes/mysql-data`

Docker Networking

- Docker networking is the configuration layer that enables isolated containers to communicate with each other.

Built-in Network Drivers

- Bridge (Default) : Via IP (Default bridge) or DNS/Container Name (User-defined).
- Host : Shares the host machine's network stack directly; no isolation.
- Overlay : Cross-host container-to-container routing without host-level changes.
- Macvlan / Ipvlan : Assigns a physical MAC/IP address to the container
- None : Completely Isolated

Crucial CLI Commands

List all active Docker networks on the host machine
`docker network ls`

Create a new custom private network using the bridge driver
`docker network create my-custom-net`

Run a container attached directly to your custom network
`docker run -d --name web-app --network my-custom-net nginx`

Connect an already running container to an existing network
`docker network connect my-custom-net database-container`

Inspect a network to look up subnets and connected container IPs
`docker network inspect my-custom-net`

Safely purge all completely unused networks from the system
`docker network prune`

Dockerfile

- A Dockerfile is a text file that contains a set of instructions for building a Docker image.
- Think of it as a recipe.
- Dockerfile → Recipe
- Docker Image → Prepared meal
- Docker Container → Person eating the meal (running application)

Why Do We Need a Dockerfile?

- Imagine you want to create an application environment.
- Without a Dockerfile, every time you would need to:
 - Install Ubuntu
 - Install Python
 - Install Flask
 - Copy your application
 - Start the application
- This is repetitive and error-prone.
- With a Dockerfile, you write the steps once, and Docker can build the image consistently every time.

Your First Dockerfile

```
FROM ubuntu  
RUN apt update  
CMD ["echo", "Hello Docker!"]
```

```
FROM alpine  
WORKDIR /myapp  
COPY . .  
RUN apk add --no-cache nodejs npm  
CMD ["node", "app.js"]
```

Understanding Each Instruction

- FROM - specifies the base image
- RUN - executes commands while building the image
- COPY - Copies files from your computer into the image
- WORKDIR - Sets the working directory
- CMD - Defines the default command when the container starts
- EXPOSE - Documents which port the application listens on
- ENV - Creates environment variables.
- ENTRYPOINT - Specifies the executable that always runs when the container starts. Fixed

Instruction	Purpose	Example
FROM	Base image	FROM ubuntu:22.04
RUN	Execute command during build	RUN apt update
COPY	Copy files from host	COPY . .
ADD	Copy files and extract archives or download from URLs (less commonly used)	ADD app.tar.gz /app
WORKDIR	Set working directory	WORKDIR /app
ENV	Set environment variable	ENV PORT=5000
EXPOSE	Document container port	EXPOSE 8080
CMD	Default command	CMD ["python", "app.py"]
ENTRYPOINT	Main executable	ENTRYPOINT ["python"]



docker tag (Targeting the Destination)

- Docker images need a specific naming format so Docker knows exactly where to upload them.
- `[registry_url/][username_or_organization]/[repository_name]:[tag_version]`
 - Registry URL: Optional. Defaults to Docker Hub if left blank. Examples: ghcr.io, quay.io, ://amazonaws.com.
 - Username/Org: Your account namespace on that registry.
 - Repository: The specific name of your application image.
 - Tag: The version identifier (e.g., v1.0, latest). Defaults to latest if omitted.

Example

- `docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]`
- `docker tag my-local-app:latest myusername/my-web-app:v1.0`

docker login (Authentication)

- # Log into official Docker Hub (prompts interactively for Username and Password)
- docker login

docker push (The Upload)

- Once tagged and authenticated, the docker push command uploads your local image layers to the target remote registry.
- `docker push TARGET_IMAGE[:TAG]`
- `docker push myusername/my-web-app:v1.0`

Flow

Step 1: Build your local image

```
docker build -t my-local-app .
```

Step 2: Authenticate with the remote registry

```
docker login
```

Step 3: Tag your image with your registry username and version

```
docker tag my-local-app myusername/my-web-app:v1.0
```

Step 4: Push the image up to the cloud

```
docker push myusername/my-web-app:v1.0
```

Docker Compose

- Docker Compose is a tool for defining and running multi-container applications.
- Instead of running complex, multi-line docker run commands for every individual service, you describe your entire application stack in a single configuration file and manage it using simplified commands.

Why Do We Need It?

- Imagine building a modern web application that requires a **frontend web server**, a **backend API**, a **database**, and a **caching layer**.
- Without Docker Compose, you would have to manually:
 - Open multiple terminal tabs.
 - Run separate docker run commands with complex parameters for ports, networks, and storage.
 - Carefully remember the exact startup order so your API does not crash while waiting for the database.
- Docker Compose solves this by turning your infrastructure into a single, version-controlled blueprint.

How Docker Compose Works

- The entire workflow relies on three core steps:
 - Define the environment: Create a Dockerfile for any custom application images.
 - Define the services: Specify all the components (database, web servers, networks, volumes) in a compose.yaml (or docker-compose.yml) file.
 - Launch the stack: Run a single command to spin up the entire application seamlessly.

What a Compose File Looks Like

services:

web-server:

image: nginx:alpine

ports:

- "8080:80"

Core Components within the Blueprint

- **Services:** The containers that make up your app (e.g., web and redis).
- **Ports:** Maps a port on your physical computer to a port inside the container.
- **Depends_on:** Forces Docker to start the redis container before launching the web container.
- **Volumes:** Used to map folders so your database data isn't deleted when a container stops.
- **Networks:** Automatically connects your containers together so they can safely speak to each other using their service names.

Essential Commands

- `docker compose up` – Downloads images, creates networks, and starts your whole application stack.
- `docker compose up -d` – Starts your app in "detached" mode so it runs quietly in the background.
- `docker compose down` – Stops and safely deletes all running containers and networks belonging to that project.
- `docker compose ps` – Shows a quick status overview of your currently running containers.
- `docker compose logs` – Consolidates and displays the live output streams from all running services in one window.

SQL

```
CREATE TABLE IF NOT EXISTS users (  
id INT AUTO_INCREMENT PRIMARY KEY,  
name VARCHAR(255) NOT NULL  
);
```

```
INSERT INTO users (name) VALUES ('Alice'), ('Bob'), ('Charlie');
```

server.js

services:

api:

image: suryakants/expressapp:latest

ports:

- "3000:3000"

depends_on:

db:

condition: service_healthy

db:

image: mysql:8.0

ports:

- "3306:3306"

environment:

MYSQL_ROOT_PASSWORD: root

MYSQL_DATABASE: myapp

volumes:

- mysql_data:/var/lib/mysql

volumes:

mysql_data:

Thank You

suryakant.surve@vinsys.com